

are there better nlp libraries than spacy and spark-nlp

Whether a library is "better" than [spaCy](#) or Spark NLP entirely depends on your specific use case, as **there is no single superior library, but rather distinct tools optimized for different tasks**. spaCy is the industry standard for fast, single-machine production pipelines, while Spark NLP rules large-scale distributed big data environments.  Databricks +4

However, several libraries outperform them if your focus shifts to cutting-edge deep learning accuracy, Large Language Model (LLM) orchestration, multilingual research, or lightweight prototyping. 

1. Hugging Face Transformers (Best for State-of-the-Art Deep Learning & LLMs)

If your primary metric for "better" is absolute accuracy on complex linguistic tasks, Hugging Face Transformers is the undisputed leader.  Reddit · r/LanguageTechnology +1

- **Why it's better:** It grants direct access to thousands of pre-trained state-of-the-art transformer architectures (like BERT, RoBERTa, and GPT variations). While spaCy and Spark NLP have added transformer support, Hugging Face is the native ecosystem where these models are born and optimized.  Medium · Allan Hall +4
- **When to choose it:** For custom sequence-to-sequence tasks, advanced text generation, or pushing the absolute limit on accuracy for tasks like Named Entity Recognition (NER) and sentiment analysis.  omophub.com +1

2. [Haystack by deepset](#) (Best for Search, RAG, and Agentic AI)

Traditional libraries treat text processing as isolated tasks (e.g., tokenisation or parsing). Haystack treats NLP as an end-to-end system.  Medium · Yash J... +1

- **Why it's better:** It is designed specifically to build Retrieval-Augmented Generation (RAG) pipelines, semantic search engines, and question-answering systems. It seamlessly connects vector databases, embedding models, and LLMs.
- **When to choose it:** If you are building a modern search application, a chatbot powered by enterprise data, or an AI agent.  omophub.com +2

3. [Stanza by Stanford NLP](#) (Best for Multilingual & Academic Accuracy)

Stanza is the official Python evolution of the legendary Stanford CoreNLP toolkit.  Anaconda

- **Why it's better:** It features an incredibly accurate, fully neural pipeline optimized consistently across more than 60 human languages. Unlike many commercial engines that treat non-English languages as an afterthought, Stanza delivers highly precise grammatical analysis and dependency parsing globally.  Anaconda +1
- **When to choose it:** For academic research, linguistic exploration, or enterprise applications that require deeply accurate multilingual text analytics.  omophub.com +1

4. [Flair](#) (Best for State-of-the-Art Sequence Labelling)

Developed by Humboldt University, Flair is a simple but powerful deep learning framework built on PyTorch.  omophub.com +3

- **Why it's better:** It uses unique "character-level" contextual string embeddings. This allows it to handle misspelled words, out-of-vocabulary terms, and subtle contextual nuances better than standard word-level models.  Reddit · r/LanguageTechnology +4
- **When to choose it:** If your main goal is building the most accurate possible Named Entity Recognition (NER) or Part-of-Speech (PoS) tagging model.  omophub.com +2

Summary Comparison Table

Library  Reddit · r/LanguageTechnology +8	Core Strength	Best Fit For...
Hugging Face	SOTA deep learning models	Advanced model customization & text generation
Haystack	Production LLM orchestration	RAG, semantic search, and AI agents
Stanza	High linguistic precision	Multilingual data & academic research

Flair	Contextual character embeddings	Advanced sequence labelling / NER
spaCy	Blazing-fast execution speed	CPU-bound production APIs & pipelines
Spark NLP	Massive distributed scaling	Terabyte-scale big data cluster computing

If you share a few details about **what specific task you are building** (e.g., semantic search, a chatbot, or high-speed data extraction) and **the scale of your text data**, I can point you toward the exact library that fits your stack.



33 sites

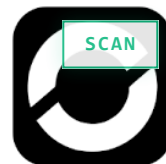
What are best NLP libraries to use with Spark - Databricks Community

16 June 2021 — By far the most popular and comprehensive library, to my knowledge, for Spark-native distributed NLP, is spark-nlp from John Snow ...



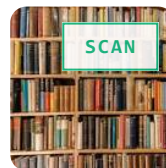
10 Key Python Libraries for NLP: A 2026 Practical Guide

6 May 2026 — Table_title: Top 10 Python NLP Libraries Comparison
Table_content: | Solution | Core focus | Target audience | | --- | --- | --- |



NLP Resources and Libraries - Medium

14 Feb 2022 — NLP Resources and Libraries * NLTK (Natural Language Toolkit) [cheatsheet] - used for building Python programs that work with huma...


[Show all](#)

Which is best for legal nlp?

For legal NLP, [Spark NLP](#) is the clear market leader for commercial, out-of-the-box enterprise applications, while Hugging Face dominates for custom, cutting-edge AI development.  John Snow Labs +1

Legal documents feature highly specialized terminology, nested clause structures, and dense paragraphs. Standard NLP libraries trained on news or web text struggle with this format.  John Snow Labs

Selecting the right tool for legal text requires balancing your engineering scale against your customization needs:

1. Spark NLP by John Snow Labs (Best for Commercial Out-of-the-Box Legal Tech)

Spark NLP features a dedicated commercial extension explicitly named **Legal NLP**. It is widely considered the industry standard for production-level enterprise legal document processing.  John Snow Labs +2

- **Pre-trained Legal Power:** It offers hundreds of ready-to-use models trained specifically on legal corpora (NDAs, loan agreements, court opinions, filings).
- **Key Capabilities:** It excels at complex legal Named Entity Recognition (extracting parties, dates, governing laws), long-span legal clause extraction, and relation extraction (e.g., tying a specific obligation to a specific party).  John Snow Labs
- **Scale:** Because it sits on top of Apache Spark, it is the only library capable of processing millions of dense legal documents concurrently across distributed server clusters.  John Snow Labs +2
- **Drawback:** The specialised legal modules require a paid commercial license from John Snow Labs.  Medium · David Cecchini

2. Hugging Face Transformers (Best for Open-Source Custom Modeling & LLMs)

If you want to build or fine-tune your own models without paying commercial license fees, Hugging Face is your best platform.  Anaconda +1

- **Access to Legal LLMs:** The platform hosts specialized, open-source domain models like **Legal-BERT**, **SauILM** (a 7B parameter model purpose-built for the legal domain), and various fine-tuned LLaMA/Mistral models optimized for legal text. 
- **Flexibility:** It is unmatched if you need to build custom workflows for legal text summarisation, contract question-answering (QA), or judicial outcome

prediction using deep learning.  Medium · Moez Ali

- **Drawback:** It requires significant machine learning expertise and GPU infrastructure to fine-tune and serve these heavy transformer models.  Reddit · r/LanguageTechnology

3. [spaCy](#) (Best for Lightweight Prototyping & Custom Rule Building)

spaCy does not provide deep, out-of-the-box legal models like Spark NLP, but it is highly valuable for hybrid engineering pipelines.  Anaconda

- **Rule-Based & Pattern Matching:** Legal text relies heavily on precise formatting and structured citations (e.g., "*15 U.S.C. § 78a*"). spaCy's EntityRuler allows you to seamlessly combine statistical models with deterministic, regular-expression-based rules to catch these patterns with 100% precision.
- **Speed and Preprocessing:** It remains the best library for quickly cleaning, tokenizing, and parsing sentences before passing them into heavier LLM systems.  Medium · Muhammad Zaky Zulfikar +1

Selection Strategy

- Choose **Spark NLP** if you are an enterprise team that needs production-grade, pre-trained legal pipelines and has the budget for commercial tooling.
- Choose **Hugging Face** if you want to deploy specialized open-source legal AI models or build a custom Retrieval-Augmented Generation (RAG) tool using legal databases.
- Choose **spaCy** if you need a lightweight, fast, free tool to extract heavily structured data (like regulatory citations or specific boilerplate patterns) using custom rules.  Medium · David Cecchini +3

If you share a few details about your primary goal—such as **contract review**, **legal research**, or **compliance auditing**—I can recommend the specific models or architecture you should use.

AI responses may include mistakes. For legal advice, consult a professional. [Learn more](#)

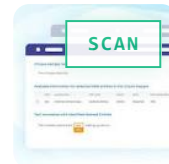




10 sites

Legal NLP - Natural Language Processing for Legal Documents

The most widely used NLP library in Legal. As with last year's survey, Spark NLP is the most popular library for production use. G...



John Snow Labs

Legal NLP 2.0: Bringing fresh capabilities for Legal research and ...

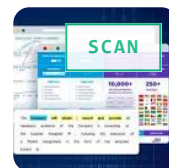
12 July 2024 — John Snow Labs is an award-winning AI and NLP company providing state-of-the-art software, models, and data to help healthcare and...



Medium · David Cecchini

Long-span clause extraction for Legal NLP using Q&A and ...

17 Nov 2022 — Spark NLP. John Snow Labs Spark NLP is the only natively scalable NLP library, since it runs on the top of Spark Clusters. It's co...



John Snow Labs

Show all

Are there features that either spark-nlp or spacy have, that the other doesn't have?

Yes, [spaCy](#) and Spark NLP have fundamentally different architectural philosophies, leading to exclusive features that the other cannot natively match. Medium · Mustafa Aytuğ Kaya

spaCy is built on a single-machine, object-oriented approach designed for developer developer workflows, whereas Spark NLP is built on Apache Spark's distributed, tabular ecosystem designed for large scale computing. StackShare

Features Exclusive to Spark NLP

Because Spark NLP is built for enterprise big-data and healthcare environments, it has out-of-the-box features that spaCy does not support without extensive manual code customisation: Domino Data Lab +2

- **Entity Resolution (Normalization):** Maps an extracted entity string directly to a unique code in a knowledge base (e.g., matching the phrase *"heart attack"* to the formal medical code `ICD-10: I21`).
- **Assertion Status Detection:** Analyzes the context of an entity to determine if it is present, absent, hypothetical, or conditional (e.g., distinguishing *"Patient denies cough"* from *"Patient has a cough"*).
- **De-identification:** Automatically detects, masks, or obfuscates personally identifiable information (PII) or protected health information (PHI) via structured obfuscation configurations.  Domino Data Lab
- **Built-in OCR Engine:** Features native integration with optical character recognition engines, allowing you to pass PDF files or raw image documents directly into the NLP pipeline.  Spark NLP
- **Distributed Cluster Scaling:** Processes terabytes of text concurrently across multi-node server clusters using native **Spark DataFrame lazy evaluation**.  Microsoft Learn +1
- **Native MLflow Tracking:** Integrates automatically with enterprise data workflows like MLflow to track parameters, versions, and deployment registries natively.  Microsoft Learn

Features Exclusive to spaCy

spaCy focuses heavily on runtime ease-of-use, custom rule engineering, and interactive development, offering unique developer

tools:  YouTube · freeCodeCamp.org +1

- **The `Doc` Object & Intrinsic Hashing:** Text is processed into an object-oriented `Doc` structure. Every string token is implicitly mapped to a 64-bit integer hash. This permits operations at speeds near C-level without maintaining full string memory arrays.
- **Built-in Visualizers (`displaCy`):** Offers a native UI module (`displaCy`) that renders interactive dependency parse trees and entity highlighting directly inside Jupyter Notebooks or web apps.
- **EntityRuler & Token Matcher:** Features an integrated system to mix raw machine-learning predictions with deterministic token-pattern matching, regular expressions, and dictionaries simultaneously.
- **Reproducible Pipeline Configs:** Utilizes a unified `.cfg` configuration blueprint that controls hyperparameters, dependencies, and tokenisation rules to prevent silent environment differences.

- **spaCy Projects:** Includes a built-in workflow runner (resembling a data-science `Makefile`) to automate data ingestion, training steps, evaluation, and remote dataset locking via cryptographic file hashing.  LinkedIn · James Hall +5

Structural Feature Comparison

Capability  YouTube · codebasics +5	spaCy	Spark NLP
Data Format	Object-Oriented (<code>Doc</code> , <code>Token</code> , <code>Span</code>)	Tabular Schema (Spark DataFrames)
Rule-ML Mix	Inline (Seamless via <code>EntityRuler</code>)	Sequential (Requires split pipeline nodes)
Deployment	Python API endpoints or serverless functions	Data clusters / Distributed execution stacks
Execution	CPU-optimised Cython / Local GPU acceleration	PySpark parallel compute engines

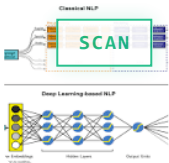
If you want to choose between them, let me know **where you intend to deploy your project** (e.g., AWS Lambda, a local server, or a Databricks/Hadoop cluster) and **what your input data formats look like**.



 12 sites

Python NLP libraries: spaCy vs NLTK vs Spark NLP
7 Apr 2019 — Stanford sells commercial licenses for CoreNLP which are required for commercial use of the library. Commercial licenses and suppo...

 Domino Data Lab



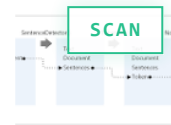
What tools, software, etc. would you say are essential ... - Reddit

9 Apr 2020 — * NLTK. I think tokenizer/filter flexibility is super under-appreciated. A lot of people seem to take for granted that tokenizers ...

 Reddit · r/LanguageTechnology

Choose a Natural Language Processing Technology - Azure ...

21 Apr 2026 — Manage models by using MLflow * Spark NLP uses Spark MLlib pipelines with native MLflow support. You don't need to write custom se...



 Microsoft Learn

If I'm using these for purely non-commercial purposes and without any funding, can I achieve a better outcome using spark-nlp from scratch, without their custom commercial models? These are my projects and datasets:

<https://huggingface.co/datasets/edithatogo/nz-hansard-corpus>

<https://github.com/edithatogo/corpus-nz-hansard>

<https://huggingface.co/datasets/edithatogo/nz-hansard-source-archive>

<https://huggingface.co/datasets/edithatogo/nz-legislation-corpus>



For your non-commercial, 6.5GB New Zealand legal datasets, utilizing Spark NLP from scratch is less efficient than combining Hugging Face with spaCy, which provides better outcomes through pre-trained legal models and rule-based extraction. While Spark NLP's open-source version can build a custom NER pipeline, it requires extensive manual annotation to match the performance of legal models already familiar with NZ law. A superior, free approach involves using spaCy's EntityRuler for the structured legislation corpus and fine-tuning legal BERT models via Hugging Face for the conversational Hansard data. [↗](#)

AI can make mistakes, so double-check responses



 1 site

State of the art Clinical Named Entity Recognition in Spark ...

1 Sept 2020 — below now i'd like to welcome vicel koiiman we'll be



What if I'm going to actually fine-tune my own models on NL law?

NZ law

If you are going to commit to fine-tuning your own models from scratch on New Zealand law and parliamentary debates, **Hugging Face is the clear winner for your project**, while **spaCy is your best supporting tool**.

For a well-resourced commercial entity, Spark NLP's open-source core *can* train custom models, but for an independent, unfunded researcher working on the **~6.5 GB NZ Hansard and Legislation corpora**, Spark NLP introduces massive architectural overhead without any performance benefit.

Why Hugging Face is the Best Choice for Fine-Tuning

When training a model to understand the nuances of New Zealand law (such as Māori legal concepts like *tikanga*, or the distinct structure of NZ statutes), you need maximum control over the modern deep learning architecture.

- **Access to Legal Foundations:** You do not have to train a model from absolute zero. You can take an open-access foundation model like **SauLLM-7B** (specifically designed for law) or **Legal-BERT**, and fine-tune it directly on your `nz-legislation-corpus` and `nz-hansard-corpus`.
- **The Hugging Face Ecosystem:** Libraries like `transformers`, `peft` (for Parameter-Efficient Fine-Tuning, which lets you train large models on a single consumer GPU), and `trl` are the global standard. 99% of new research, tutorials, and optimization techniques are built for Hugging Face.
- **Tokenization Customization:** NZ legal text and Hansard debates contain specific vocabulary (e.g., *Aotearoa*, specific act naming conventions, parliamentary terms). Hugging Face allows you to easily expand the tokenizer's vocabulary and train it specifically on your data so the model doesn't break these unique terms into meaningless sub-words.

Where spaCy Fits In (Your Data Preparation Engine)

Do not abandon spaCy entirely. Before you can fine-tune a model on Hugging Face, you need to clean your data and create training labels (especially for Named Entity Recognition or Clause Extraction).

- **Label Generation via Rules:** You can use spaCy's `EntityRuler` and regex features to automatically find and label millions of statutory citations, dates, MP names, and section breaks in your raw text.
- **Exporting to Hugging Face:** Once spaCy programmatically labels your corpus, you can easily convert those spans into token-classification datasets to feed directly into a Hugging Face training pipeline.

Why Spark NLP is the Wrong Choice for Fine-Tuning from Scratch

While Spark NLP excels at running inference on massive datasets using *pre-existing* models, training custom deep learning models on it from scratch without their commercial suite is notoriously painful for independent developers:

1. **The "Spark Tax":** To train models in Spark NLP, you have to wrap all your data into Spark DataFrames, configure a local Spark cluster, manage Java Virtual Machine (JVM) memory heaps, and handle PySpark-to-Java serialization bottlenecks. This is a massive engineering distraction when your real goal is linguistic modeling.
2. **Hardware Inefficiency:** For an unfunded project, you will likely be training on a single GPU (or a free cloud instance like Google Colab/Kaggle). Spark NLP is engineered to scale across *clusters* of machines. For single-GPU training, native PyTorch/Hugging Face is significantly faster and uses less VRAM.
3. **Community and Troubleshooting:** If you run into a training error or a tensor mismatch in Hugging Face, a quick web search will yield hundreds of solutions. In open-source Spark NLP, documentation for custom deep-learning training workflows is sparse, as most of their enterprise users rely on their pre-baked commercial models.

Recommended Free Roadmap for Your NZ Corpora

1. **Pre-Process with spaCy:** Use spaCy to segment sentences, clean up the OCR/text anomalies in the `nz-hansard-source-archive`, and use rule-based

matching to auto-tag obvious legal entities (like "Section 4 of the Crimes Act 1961").

2. **Domain Adaptation (Optional but powerful):** Take a base model (like `nlpauweb/legal-bert-base`) and perform *Masked Language Modeling* (unsupervised training) on your entire 6.5 GB corpus so the model learns the statistical patterns of NZ legal syntax.
3. **Fine-Tune with Hugging Face:** Take that adapted model and fine-tune it on your specific target tasks (e.g., classifying text by policy topic, extracting complex legal obligations, or analyzing sentiment/intent in parliamentary speeches) using a single GPU.

To help guide your next steps, **what specific task** are you hoping to achieve with your fine-tuned model (e.g., **summarizing debates**, **extracting legal entities**, or **classifying statutes by area of law**)?

AI responses may include mistakes. For legal advice, consult a professional. [Learn more](#)



So for the combination that I'm trying to achieve, it sounds like starting with spacy and hugging face libraries, swapping high... ▼

Yes, you are effectively describing the architecture of `spacy-transformers`.

You do not need to manually write or "swap in" Rust components yourself. The standard library for combining these two worlds (`spacy-transformers`) **already runs on Rust under the hood** for its most performance-critical tasks.

By using this existing library, you get the developer experience of Python with the raw speed of Rust for the bottlenecks.

The Existing "Bridge" Library: `spacy-transformers`

This is the standard library maintained by Explosion (the creators of spaCy) that lets you use Hugging Face models (like `Legal-BERT`) directly inside a spaCy pipeline.

It achieves high performance by offloading the "heavy lifting" to two specific Rust-backed libraries:

1. `tokenizers` **(by Hugging Face)**: This is written in Rust. It handles the conversion of your 6.5 GB of text into numerical IDs. Because it is Rust-based, it is multi-core and incredibly fast (often 10x–20x faster than pure Python tokenizers). [🔗](#)
2. `spacy-alignments`: This is a library written in Rust specifically to solve the "alignment problem" (matching spaCy's linguistic tokens to BERT's sub-word tokens).

Verdict: You don't need to build a new integration. You just need to install `spacy-transformers`.

How to Implement This for Your NZ Law Project

Since you want to combine the **linguistic precision** of spaCy (for statutes/entities) with the **learned context** of Hugging Face (for parliamentary debate meaning), here is the architecture you should use:

1. The Setup

You will configure a spaCy pipeline that pulls in your fine-tuned NZ model.

python

```
# You don't need to write Rust; this library uses Rust internals automatically
# pip install spacy-transformers
import spacy

# This config object tells spaCy to use the Hugging Face transformer model
config = {
    "model": {
        "@architectures": "spacy-transformers.TransformerModel.v3",
        "name": "nlpauieb/legal-bert-base-uncased", # Replace with your model name
        "tokenizer_config": {"use_fast": True} # This flag forces fast tokenization
    }
}

nlp = spacy.blank("en")
nlp.add_pipe("transformer", config=config)
```

Use code with caution.

2. Why this is "Better" than Spark NLP for you

- **Zero Overhead:** You don't need a Java VM or a cluster manager.
- **Rust Speed:** The tokenization of your 6.5 GB corpus will happen at C/Rust speeds because `use_fast=True` invokes the Rust binary.
- **Dynamic Alignments:** When you extract a "Date" or "MP Name" using spaCy's rules, `spacy-alignments` (Rust) automatically maps that exact span to the correct vectors in the BERT model, allowing you to use those vectors for downstream tasks (like classification) without losing track of where the words are.

Advanced: If You *Really* Want Pure Rust (Hard Mode)

If your goal is purely to learn Rust or build an ultra-lightweight inference engine (not just use Python), Hugging Face does have a pure-Rust machine learning framework called [Candle](#).

- **What it is:** A minimalist ML framework that lets you run BERT/LLMs without Python or PyTorch installed.
- **When to use it:** Only if you need to deploy your NZ Law model into a tiny binary (e.g., running inside a web browser via WASM or on a Raspberry Pi).
- **Recommendation: Avoid this for now.** For fine-tuning and research, the Python ecosystem (leveraging Rust binaries) is superior.

Summary

Do not reinvent the wheel. Use `spacy-transformers`. It is the exact hybrid you asked for—Python for logic, Rust for performance—and it is the industry standard for what you are trying to build. [🔗](#)



2 sites

Huggingface Releases New Tokenizers Library Written in Rust

11 Jan 2020 — Huggingface, the NLP research company known for their popular transformers library, has now released a new library for state-of-th...

 Reddit · r/rust

Some questions about Spacy vs Hugging face transformers ...

19 Feb 2021 — You can train your own models with spacy though. And the latest spacy version I think supports transformers. • 5y ago. Yes you can...

 Reddit · r/LanguageTechnology

Outline my approach for both Legal-NLP and NZ's corpus, as well as the approach to Parliamentary NLP using this. Is ther... 

Your choice of your **NZ Legislation** and **NZ Hansard** datasets creates a hybrid project spanning two closely related but distinct domains: **Legal NLP** and **Parliamentary NLP**.

Is There a Separate "Parliamentary NLP" Project?

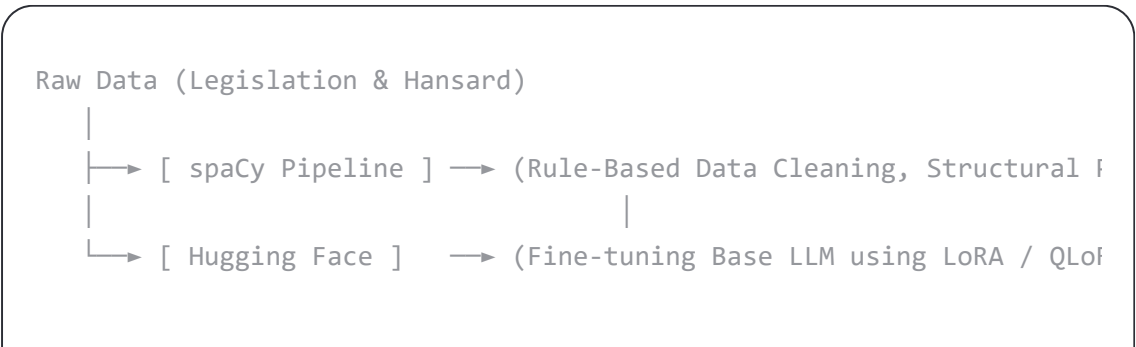
Yes, **Parliamentary NLP** is an established, independent subfield of computational linguistics, distinct from traditional Legal NLP.  ScienceDirect.com +1

- **Legal NLP** focuses on rigid, logical, structured texts like statutes, regulations, contracts, and case law. It cares about definitions, obligations, liabilities, and formal cross-references.  ar... +1
- **Parliamentary NLP (or Legislative NLP)** focuses on political discourse, rhetoric, debates, and committee transcripts. It cares about ideological positioning, sentiment, policy salience, speaker alignments, and tracing how an MP's speech eventually transforms into a written law.  ScienceDirect.com +4

Prominent global benchmarks in this space include **ParlaMint** (a massive multilingual European parliamentary corpus), [ParlaBench](#) (benchmarking LLMs for political authenticity and party alignment in the UK parliament), and [ParlaSent](#) (specialized sentiment analysis for legislative proceedings). Your `nz-hansard-corpus` fits into this global domain.  Taylor & Francis Online +1

The Unified Architecture (`spacy-transformers`)

To process both datasets without cloud funding, deploy a split strategy unified under `spacy-transformers` (leveraging Python for syntax and Rust for fast tokenization).



[Unified Pipeline]

Phase 1: The Legal-NLP Approach (`nz-legislation-corpus`)

Statutes are structured, logical, and literal. The objective here is to teach the model **structural logic** and **citation networks**. [✖ ar... +1](#)

1. Pre-Processing & Structural Partitioning (spaCy)

- **Goal:** Chop massive Acts into micro-units (Parts, Sections, Clauses, Subsections).
- **Action:** Use spaCy's `Matcher` or regular expressions to isolate layout anchors (e.g., matching lines starting with `Section \d+` or `Part [A-Z]+`).
- **Outcome:** A structured database where every line of text is tied to its precise hierarchy in NZ Law. [✖ arXiv](#)

2. Legal Entity Extraction (spaCy EntityRuler)

- **Goal:** Tag formal entities with 100% precision without wasting GPU power.
- **Action:** Build rules for highly predictable legal patterns like statutory cross-references (e.g., "*within the meaning of section 5 of the Electoral Act 1993*") or definitions ("*In this Act, **public notice** means...*"). [✖ arXiv](#)

3. Domain Adaptation (Hugging Face)

- **Goal:** Make a base model understand vocabulary specific to New Zealand (such as *Te Reo Māori* legal definitions like *tikanga* or *taonga* embedded in modern statutes).
- **Action:** Take an open-access legal model like **SauLM-7B** or **Legal-BERT**. Run **Masked Language Modeling (MLM)** natively using your raw `nz-legislation-corpus`. This requires zero manual labeling and teaches the model how NZ legal sentences are constructed. [✖ arXiv](#)

Phase 2: The Parliamentary NLP Approach (`nz-hansard-corpus`)

Hansard transcripts are conversational, chaotic, ideologically driven, and unstructured. The objective here is to track **people, topics, and**

sentiment.  Zenodo +2

1. Speaker & Metadata Alignment (spaCy)

- **Goal:** Correctly attribute who is speaking and map them to their political party, electorate, and role (e.g., Backbencher vs. Minister).
- **Action:** Clean the `nz-hansard-source-archive` using spaCy to isolate the speaker identifier tokens (e.g., `Hon Dr DUNCAN WEBB (Minister for...):`). Use these anchors to split transcripts into individual speaker "turns."

2. Fine-Tuning Task-Specific Models (Hugging Face)

Because you have zero budget, use **QLoRA via Hugging Face** `peft` to train a lightweight version of a small LLM (like Llama-3-8B or Mistral-7B) on a free Google Colab or Kaggle GPU.

- **Task A: Topic Modeling / Categorization:** Fine-tune your model to assign a policy label (e.g., *Housing, Taxation, Treaty Issues*) to Hansard text blocks.
- **Task B: Political Sentiment & Positioning:** Fine-tune the model to measure ideological positions or track specific policy issues (like the "Just Transition" index used globally in parliamentary analytics).  ScienceDirect.com +1

Phase 3: The Cross-Domain Link (Linking Hansard to Legislation)

The ultimate execution of your project comes from **connecting the two datasets**.

1. Use your **spaCy Legal-NLP pipeline** to scan the **Hansard corpus** for mentions of specific bills or existing Acts.  arXiv
2. When an MP says, "*We must amend Section 4 of the Crimes Act*," spaCy automatically flags `Crimes Act 1961` and links it directly to the exact text inside your `nz-legislation-corpus`.
3. This creates a data graph: you can track exactly which political parties (from Hansard) debated, opposed, or championed specific clauses of a bill that eventually became an enacted law.  arXiv

How far along are you with **downloading and cleaning the raw source archive texts**? If you have a snippet of raw text from either dataset, I can show

you how to write the initial spaCy processing script for it.



10 sites

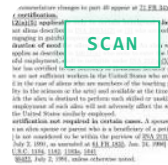
Natural Language Processing for the Legal Domain - arXiv

This survey article explores the current landscape of NLP applications within the legal domain. It discusses its potential benefit...



Natural Language Processing for the Legal Domain: A Survey of ...

6 Dec 2025 — 3.2 Legal NLP. The legal sector has been exploring AI-driven solutions since the late 20th century, applying NLP techniques to aut...



ACM Digital Library

Using Natural Language Processing to Analyze Political Party ...

1 Mar 2023 — Abstract. This study explores how natural language processing (NLP) can supplement content analyses of political documents, partic...



MDPI

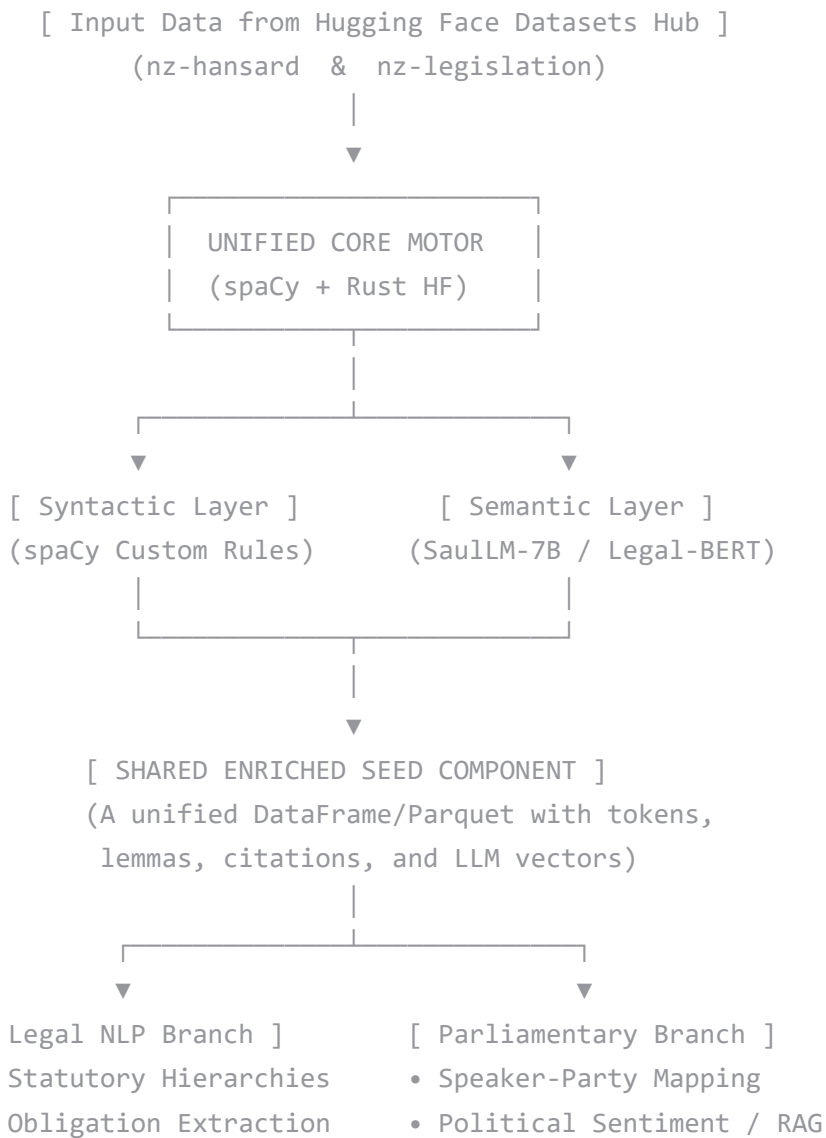
Show all

I want a unified core that is bleeding edge, will allow the same approach and as many shared features to be extracted, before moving in separate directions. I already have these projects setup in hugging face, github and zenodo, so I'll want to build on this.



To build a **unified core** that is bleeding-edge, highly optimized, and shared across both datasets, you should architect a pipeline that uses **spaCy as the programmatic data coordinator** and **Hugging Face (transformers + peft)** for the neural semantic embeddings.

Because your datasets are already hosted on Hugging Face and Zenodo, you can pull them directly into a unified preprocessing format before splitting them into task-specific applications downstream.



Step 1: The Unified Preprocessing Engine

The goal is to push both the Hansard data and the Legislation text through the exact same processing script. This ensures that every document, whether it is a speech by an MP or a subsection of an Act, shares an identical baseline schema.

Create a single script leveraging Hugging Face's **Rust-backed Fast Tokenizers** and spaCy's streaming pipeline (`nlp.pipe`) to handle the datasets in parallel chunks without running out of RAM.

```
python
```

```
import spacy
from datasets import load_dataset
```

```

# 1. Load your existing HF datasets directly
hansard_ds = load_dataset("edithatogo/nz-hansard-corpus", split="train")
legislation_ds = load_dataset("edithatogo/nz-legislation-corpus", split="train")

# 2. Build a unified blank core with custom pipeline components
nlp = spacy.blank("en")

# Add a boundary detector to handle long legal clauses and hours of sp
nlp.add_pipe("sentencizer")

# 3. Create a shared data extraction function
def extract_unified_features(batch):
    texts = batch["text"] # Assumes your HF datasets use a 'text' column

    # Process text using spaCy's optimized streaming (uses C-level ops)
    docs = list(nlp.pipe(texts, batch_size=256, n_process=-1)) # -1 uses all CPUs

    cleaned_tokens = []
    sentence_counts = []

    for doc in docs:
        # Shared Feature 1: Clean, lowercase tokens ignoring punctuation
        tokens = [token.text.lower() for token in doc if not token.is_punct]
        cleaned_tokens.append(tokens)

        # Shared Feature 2: Standard structural length
        sentence_counts.append(len(list(doc.sents)))

    batch["cleaned_tokens"] = cleaned_tokens
    batch["sentence_count"] = sentence_counts
    return batch

# Apply identically to both datasets
processed_hansard = hansard_ds.map(extract_unified_features, batched=True)
processed_legal = legislation_ds.map(extract_unified_features, batched=True)

```

Use code with caution.

Step 2: The Shared Feature Extraction Layer

Before you branch off into political science or legal analytics, both datasets must yield the same foundational attributes. Your core should extract the following simultaneously:

- python

Use code with caution.

To integrate modern deep learning without cloud funding, map your processed dataset tokens to shared dense vector spaces using a free local machine learning pipeline.

- **The Model:** Use `Equal1/SauLM-7B` or `nlpaueb/legal-bert-base-uncased`. Even though SaulLM is a 7B parameter model, you can run it in a 4-bit quantized state locally using `bitsandbytes` for zero dollars.

- https://www.google.com/search?q=are+there+better+nlp+libraries+than+spacy+and+spark-nlp&sca_esv=8df6689b8d0e1c8b&rlz=1C1GCEA_e... 21/40

python

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig

# 4-bit quantization configuration for consumer/free GPUs
quant_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16
)

# Load a bleeding-edge legal open-source model
model_id = "Equall/SauLLM-7B"
tokenizer = AutoTokenizer.from_pretrained(model_id, use_fast=True) # l
model = AutoModelForCausalLM.from_pretrained(model_id, quantization_c
```

Use code with caution.

By generating these embeddings on your pre-processed corpora, you unlock the ability to do semantic similarity searches *across both datasets* (e.g., finding the exact section of law that matches a vaguely worded complaint by an MP during a reading).

Step 4: The Separate Directions (Downstream Tasks)

Once your data is cleaned, feature-extracted, and vector-embedded by the core engine, you can export it to your GitHub/Hugging Face project branches as a unified `.parquet` file. From there, you branch off:

Direction A: Parliamentary NLP

- **Input:** Enriched Hansard dataset.
- **Downstream Action:** Filter by the `NZ_ACT` entities extracted by the core. Group the data by your speaker metadata to run sentiment analysis or use Hugging Face `peft` to fine-tune a model predicting party alignments based on how they debate specific bills.

Direction B: Legal NLP

- **Input:** Enriched Legislation dataset.

- **Downstream Action:** Use the structural layout parsing (Parts, Sections) to build an interactive network graph showing which modern NZ laws reference older laws, or train a token-classification model to isolate binding legal obligations ("must", "shall") from optional provisions ("may").

How to Proceed With Your Existing Infrastructure

Since your repos are live on Hugging Face, your next step is writing a GitHub Action or local training loop script that pulls directly from [edithatogo/nz-hansard-source-archive](#), runs this unified spaCy-Rust-Tokenizer transformer flow, and pushes the newly created feature columns back up to your Hugging Face Datasets Hub.

Would you like assistance setting up the **GitHub Actions workflow automation** or writing the explicit **Māori token preservation regex patterns** to ensure your spaCy core protects *Te Reo* legal terminology during tokenization?



Does it require vectorization or creating a graph database, or anything like that, first?

No, you do not need to set up an external vector database (like [Pinecone](#), Milvus, or [Qdrant](#)) or a graph database (like Neo4j) to get this unified core running. 

In fact, introducing those external database systems too early will stall your progress with massive engineering overhead. Because you are working with a finite, historical dataset (~6.5 GB), you can achieve high-performance results using local file formats and local memory.

Here is what you actually need to do first, what you can skip, and how to structure your storage.

1. What You Can Skip (For Now)

- **No External Vector Database:** Vector databases are built to handle billions of dynamically updating data points in real time. Your corpus is static. You

can calculate your text embeddings using Hugging Face and store them locally inside your dataset files.

- **No Graph Database Software:** While your legal cross-references form a "graph" (e.g., *Hansard Debate A* references *Act B*, which references *Section C*), you do not need to install complex graph database software. You can model these connections using standard Python tables.

2. What You Should Do First: The "Data Lake" Approach

Instead of a database, your immediate priority is saving your raw text into a highly optimized, compressed, tabular file format: **Apache Parquet**.

Parquet is a columnar file format that handles text data and heavy machine-learning vectors seamlessly. It integrates natively with the Hugging Face `datasets` library, allowing you to stream gigabytes of data directly into your scripts without overloading your computer's RAM.

The Flat-Table Schema

Your unified core should loop through your raw data, apply spaCy and the Hugging Face tokenizer, and save the results into a flat table with these shared columns:

Column Name	Data Type	Description / Purpose
<code>doc_id</code>	String	A unique identifier (e.g., <code>NZ-HANSARD-2023-104</code> or <code>NZ-ACT-1961-043</code>)
<code>corpus_source</code>	String	Explicitly flags whether the row is <code>hansard</code> or <code>legislation</code>
<code>raw_text</code>	String	The unaltered raw text snippet (a speech turn or a legislative section)
<code>cleaned_tokens</code>	List[String]	Tokenized text filtered by your spaCy rules
<code>extracted_acts</code>	List[String]	Array of any NZ Acts caught by your spaCy <code>EntityRuler</code>
<code>embeddings</code>	List[Float]	Optional: The mathematical vector generated by your Hugging Face model

3. How to Build the "Graph" and "Vectors" inside Python

Instead of setting up separate software stacks, you can generate your vectors and link your dataset connections directly in memory using standard Python data science libraries.

For the Vectors (Semantic Search / Similarity)

When you run your text through your Hugging Face model, it outputs an array of numbers (the embedding vector). You append this array directly to your dataset row. If you want to search through your data later using semantic vectors, you can use **FAISS (Facebook AI Similarity Search)**. It requires no server setup and runs entirely within your Python environment.

python

```
# Hugging Face datasets has FAISS built right in. No database required
processed_dataset.add_faiss_index(column="embeddings")

# You can now perform instant semantic searches across your 6.5 GB dataset
scores, retrieved_examples = processed_dataset.get_nearest_examples(
    "embeddings", query_vector, k=5
)
```

Use code with caution.

For the Graph (Legal Cross-References)

To build your network of how politicians debate specific laws, use **NetworkX** (a lightweight Python graph network library).

Because your spaCy core extracts a list of `extracted_acts` for every row, you can map the relationships instantly by tracking which rows share the same Act names.

python

```
import networkx as nx

G = nx.Graph()

# If an MP mentions the Crimes Act in a Hansard row, create a connecti
```

```
for row in processed_dataset:
    if "Crimes Act 1961" in row["extracted_acts"]:
        # Connects the Hansard speech ID directly to the Legislation ,
        G.add_edge(row["doc_id"], "NZ-ACT-1961-043")
```

Use code with caution.

Summary of Your Immediate Next Steps

1. **Do not install databases.** Do not download Neo4j or sign up for a cloud vector provider.
2. **Keep it in Parquet.** Use your existing Hugging Face repository to store your processed data as `.parquet` files.
3. **Run the Unified Core.** Write your script to populate the flat table layout shown above.

Once your pipeline creates that single, enriched table containing text, structural tokens, and extracted legal entities, you will have a rock-solid foundation.

Would you like to write the **spaCy function that generates unique structural IDs** (like tracking Parliament sessions or Statute sections) so your rows link up flawlessly?



1 site

Beyond the Box: Exploring Knowledge Graphs for Semantic Search, Syntactic Search, and Graph Search

8 Sept 2024 — Graph Databases: Specialized databases like Neo4j, Amazon Neptune, and Janus Graph that support graph data models.

 Artificial Intelligence in Plain English

Can you provide a brief xml coding agent to the local agent which is working to standardise the two repo's, so that it ...



Below is a complete, self-contained **XML Coding Agent specification**. You can copy and paste this system prompt directly into your local LLM or orchestration agent framework.

This agent is explicitly designed to standardise your two repositories (`nz-legislation-corpus` and `nz-hansard-corpus`) by enforcing a shared, unified data pipeline before branching out into domain-specific Legal and Parliamentary NLP features.

Agent System Prompt: The NZ-Corpus Unified Standardization Agent

```
markdown

ROLE: Expert Data Engineer and NLP Architect specializing in New Zealand

OBJECTIVE:
Your core objective is to write the Python modules that process, clear, and
1. `nz-legislation-corpus` (Highly structured, hierarchical statutory
2. `nz-hansard-corpus` / `nz-hansard-source-archive` (Conversational,

You must architect a "Unified Core Engine" workflow. This means both i

CORE DESIGN PRINCIPLES:
1. "Shared First": Minimize code duplication. If a function can proces
2. No Heavy Infra: Do not use external graph or vector databases. Rely
3. Bleeding-Edge Speed: Leverage spaCy v3 components alongside Hugging

XML CODING TASK PROTOCOL:
When the user asks you to implement a stage of this standardization p

<architecture_plan>
Explain the mathematical, linguistic, or structural strategy being app
</architecture_plan>

<unified_core_code>
```python
Place fully functional, production-ready, side-effect-free Python co
Include typing, docstrings, streaming mechanisms, and memory-safety
...
</unified_core_code>

<schema_validation>
Provide a concise Markdown table mapping out exactly how the output co
```

```
</schema_validation>
```

```
<downstream_branching>
```

Detail how the downstream code can safely consume this specific output

```
</downstream_branching>
```

Use code with caution.



## Immediate Execution Example: Defining the Unified Core Base

To kick off your local agent using this new workflow protocol, issue the prompt below. It will generate your foundational text ingestion script using the strict XML format.

**Copy and paste this test prompt to your local agent:**

text

Run Task 1: Generate the baseline ingestion script for our unified core. Load a small batch of both 'edithatogo/nz-legislation-corpus' and 'ed: Process them through a shared blank spaCy pipeline that uses the fast Output a standardized local flat structure containing doc\_id, corpus\_ Follow your XML protocol exactly.

Use code with caution.

Would you like to extend this agent template with **explicit Te Reo Māori tokenisation exceptions** or **custom regex rules for catching NZ statutory cross-references** before passing it to your local system?



Perhaps just start by telling them what libraries should/could be used, what the intention is, and what the core shared pipelin...

## The Strategy

This architecture synchronises your **NZ Legislation** and **NZ Hansard** corpora into a single data ecosystem. By standardising early, you ensure that every section of law and every word spoken in Parliament share the same linguistic foundation. This makes cross-referencing between them automatic.

---

## Required Libraries & Stack

To keep this pipeline cutting-edge, resource-efficient, and free from database overhead, the agent must build exclusively on these core Python and Rust-backed libraries:

- `datasets` (**Hugging Face**): To stream, map, and save your 6.5 GB corpora locally as Apache Parquet files without running out of RAM.
  - `spacy` (**v3+**): To drive the core pipeline orchestration, handle fast rule-based matching, and run sentence tokenization.
  - `transformers` (**Hugging Face**): To manage the deep learning models and leverage their multi-core **Rust-backed Fast Tokenizers** (`use_fast=True`).
  - `networkx`: To model the relational links (e.g., MP → Bill → Statute) natively in memory as a graph network.
  - `faiss-cpu`: To handle fast, local semantic vector search directly inside your tables without a database server.
- 

## Core Intention

The intention is to establish a **Unified Processing Core Engine**.

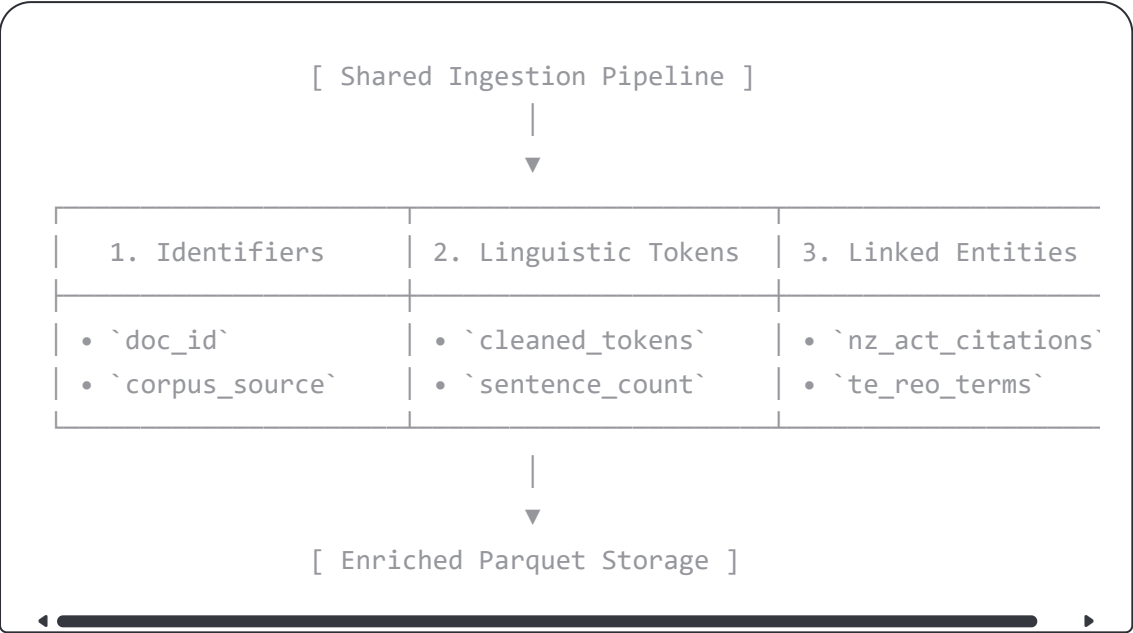
Instead of writing two different codebases, both datasets pass through an identical text-processing pipeline. This engine structures raw text into a uniform format, attaches linguistic features, extracts NZ statutory citations, and appends semantic vectors.

Once this shared step is complete, the data is saved as an enriched, unified dataset. Only *after* this step do your scripts split into separate directions for domain-specific analytics.

---

## The Shared Core Pipeline & Features

The agent must implement a pipeline that processes text chunks into a standardized layout. The shared features extracted from *both* datasets include:



1. Standardized Identifiers

- `doc_id`: A unique structural string code (e.g., `NZ-ACT-1961-043-SEC-4` for legislation or `NZ-HANS-2023-05-12-SP-04` for Hansard).
- `corpus_source`: A simple category tag (`legislation` or `hansard`) allowing downstream tasks to filter data instantly.

2. Cleaned Linguistic Tokens

- `cleaned_tokens`: Lowercased text arrays stripped of punctuation and layout whitespace, generated at C/Rust speeds.
- `sentence_count`: A metric measuring document density, ensuring long statutory clauses and multi-hour filibusters are tracked identically.

3. Linked Entities (The Cross-Domain Bridge)

- `nz_act_citations`: Extraction of specific legislation names (e.g., "*Crimes Act 1961*") using a shared spaCy `EntityRuler`. If found in Legislation, it marks a definition; if found in Hansard, it marks a political debate point.
- `te_reo_terms`: Tracking of Māori legal and cultural concepts (e.g., *tikanga*, *taonga*, *kawanatanga*) using custom tokenization exceptions to prevent modern models from breaking these words down incorrectly.



## Instructions for Your Local Agent

You can now feed this context to your local agent alongside the system prompt provided earlier.

If you are ready to begin generating the code, let the local agent know: **"Implement Stage 1: Build the unified ingestion data stream using Hugging Face datasets and the blank spaCy pipeline skeleton."**



Provide that as an XML prompt for the local coding agent.

Here is the complete, structured XML prompt ready to be copied and pasted directly to your local coding agent to initiate the unified standardization pipeline.

xml

```

<instruction_packet task="stage_1_unified_core">
 <context>
 We are building a unified core data pipeline to standardise the following datasets:
 1. 'edithatogo/nz-legislation-corpus' (Structured statutory text)
 2. 'edithatogo/nz-hansard-corpus' (Conversational parliamentary text)

 The ultimate goal is to process both datasets through an identical pipeline.
 </context>

 <technical_stack>
 - Python 3.10+
 - Hugging Face `datasets` (for streaming data loading)
 - `spacy` v3.x+ (for pipeline orchestration, sentence segmentation)
 - `transformers` with fast Rust tokenizers (`use_fast=True`) (for NLP processing)
 - `pandas`/`pyarrow` (for local Parquet generation)
 </technical_stack>

 <pipeline_specifications>
 Implement a python script or notebook execution block that accomplishes the following:

 1. Streaming Data Ingestion: Use the Hugging Face `datasets` library to stream data from the specified sources.
 2. Blended spacy Pipeline Skeleton: Instantiate a blank spaCy pipeline and load the necessary components (tokenizer, parser, ner, etc.).
 3. Feature Extraction Array: Run text batches through the pipeline to extract features.
 </pipeline_specifications>

```

```

- `doc_id`: Unique identifier string derived from metadata
- `corpus_source`: Explicit string category string ('legis
- `raw_text`: The uncleaned input string
- `cleaned_tokens`: List of lowercase word tokens, omittin
- `sentence_count`: Integer calculation of total sentences
4. Māori Language Guard: Inject custom tokenization rules into
</pipeline_specifications>

```

```
<delivery_format_protocol>
```

You must respond using the following XML tags:

```
<architecture_plan>
```

Summarize how your code structures both streams symmetrically

```
</architecture_plan>
```

```
<unified_core_code>
```

Provide clean, modular, production-ready Python code implementing

```
</unified_core_code>
```

```
<schema_validation>
```

Display a Markdown table validating the alignment of colour

```
</schema_validation>
```

```
</delivery_format_protocol>
```

```
</instruction_packet>
```

Use code with caution.

Once the agent executes this, **let me know the code it generates** or the **schema outputs** it designs so we can configure the shared legal pattern-matching engine (`EntityRuler`) next.



Would it make sense to build the pipeline as a third repo, that is then imported as a subrepo into each of the others?

Yes, **creating a third repository for the pipeline is the absolute best architectural approach** for your project.



In software engineering, this is known as the **"Shared Core"** pattern. Since you are managing two separate repositories that must stay mathematically and linguistically aligned, putting the pipeline code into its own repository prevents your codebases from drifting apart.

---

## Why a Third Repo is the Best Choice

- **Single Source of Truth:** If you change how you tokenize Te Reo Māori or update a regex pattern for NZ Acts, you only change it **once** (in the pipeline repo). Both the Hansard and Legislation repos instantly get the update.
  - **Keeps Data Repos Clean:** Your existing repos can focus purely on hosting the data files, documentation, and dataset cards, while the third repo holds the heavy Python/Rust engineering logic.
  - **Prevents Code Duplication:** Without this, you would end up copy-pasting the same spaCy and Hugging Face code into both repositories, which inevitably leads to bugs when you forget to update one side.
- 

## Git Submodules vs. Pip Packages (How to implement it)

While you *could* import it as a Git submodule, **there is an even cleaner, bleeding-edge approach** that fits perfectly with GitHub and Hugging Face: **Creating a lightweight Python package that installs directly via Git.**

Instead of managing messy Git submodule tracks, you can configure your third repository to be installed using a single `pip` command.

### 1. Structure of the Third Repo (`nz-legal-nlp-core`)

Name your third repository something like `nz-legal-nlp-core`. Its file structure should look like this:

text

```
nz-legal-nlp-core/
├── pyproject.toml # Defines dependencies (spacy, datasets, tr
├── nz_legal_core/
│ ├── __init__.py
│ └── tokenizer.py # Holds your Te Reo Māori exception rules
```

```
└─ pipeline.py # Holds the shared spaCy + HF pipeline engine
└─ patterns.py # Holds the NZ Act citation regex patterns
```

Use code with caution.

## 2. How the Other Repos Import It

Whenever you are working in your `nz-hansard-corpus` or `nz-legislation-corpus` repositories, you can install your unified pipeline engine directly from GitHub by adding this single line to your local environment or GitHub Actions script:

**bash**

```
pip install git+https://github.com
```

Use code with caution.

Then, inside your notebook or python files for either repo, you import your unified tool seamlessly:

**python**

```
This works identically inside BOTH your Hansard and Legislation scripts
import datasets
from nz_legal_core.pipeline import load_unified_core

1. Load your local repo's specific data stream
ds = datasets.load_dataset("edithatogo/nz-hansard-corpus", split="train")

2. Pull the identical engine from your 3rd repo
nlp = load_unified_core()

3. Process with 100% architectural parity
processed_ds = ds.map(lambda batch: nlp.process_batch(batch), batched=True)
```

Use code with caution.

## Update for Your XML Prompt

If you proceed with this approach, pass this quick amendment to your local coding agent before it begins writing code:

xml

```
<architecture_amendment>
```

Exclusively design the code to reside in a standalone, reusable Py

```
</architecture_amendment>
```

Use code with caution.

Would you like to generate

the `pyproject.toml` **configuration** and `__init__.py` **structure** for this third repository so you can initialize it on GitHub right away?



What should I name it?

To ensure your third repository integrates seamlessly into the global open-source ecosystem, the name should be **clear, lowercase, and explicitly declare its purpose**.

Because it is a Python library that bridges your data to NLP models, the naming conventions differ slightly between GitHub and Python.

## The Recommendation

- **GitHub Repository Name:** `nz-legal-nlp-core` (using hyphens)
- **Python Import Name:** `nz_legal_nlp_core` (using underscores)

Using hyphens for GitHub repositories makes URLs clean and highly readable (e.g., `://github.com`). Using underscores for the internal Python directory ensures it complies with Python's naming PEP 8 guidelines [1, 2], so you can type `import nz_legal_nlp_core` without syntax errors. [🔗](#)

## Alternative Options

Depending on your personal preference or future scope, here are a few refined alternatives:

Name <a href="#">↗</a>	Vibe / Purpose
<code>nz-parl-legal-core</code>	Best if you want to explicitly acknowledge the combination of <b>Parliamentary</b> and <b>Legal</b> text structures right in the name.
<code>nz-legislative-nlp</code>	A slightly more academic and precise name that covers both the <i>making</i> of laws (Hansard) and the <i>enacting</i> of laws (Legislation).
<code>parlamint-nz-core</code>	Only use this if you intend to format your Hansard data to match the global <b>ParlaMint</b> XML schema standards used across Europe.

## How to Initialize the Third Repo

When you create this repository on GitHub, create a root folder named `nz_legal_nlp_core` for your package files. Your very first configuration file (`pyproject.toml`) should look like this to establish the project:

toml

```
[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"

[project]
name = "nz-legal-nlp-core"
version = "0.1.0"
description = "Unified spaCy and Hugging Face NLP engine for New Zealand"
readme = "README.md"
requires-python = ">=3.10"
dependencies = [
 "spacy>=3.7.0",
 "transformers>=4.40.0",
 "datasets>=2.19.0",
 "accelerate>=0.30.0",
 "pyarrow>=16.0.0"
]
```

Use code with caution.

Which of those name variations resonates most with how you view the long-term goal of your project?



3 sites

#### Go Modules - Artifactory

Set the Repository Key value, and enter <https://github.com/> in the URL field as displayed below.



#### python project structure: folder naming convention

5 Sept 2018 — python project structure: folder naming convention 1 PEP 8 is a guideline. You are free to follow it or not. 1 It's more important...



#### When building in version 3.2.1 "repository name must be lowercase"

13 Mar 2021 — When building in version 3.2. 1 "repository name must be lowercase" rjkmacrocom derteufelqwe rjkmacrocom (RJK@macrocom) (derteufel...



How about `nlp_policy_nz`

`nlp_policy_nz` is an excellent name [1]. It is short, descriptive, and accurately captures the intersection of natural language processing and government policy.

To ensure it works flawlessly across both GitHub and Python, implement it with this slight structural adjustment:

- **GitHub Repository Name:** `nlp-policy-nz` (using hyphens, which is standard for URLs)
- **Python Import Package Name:** `nlp_policy_nz` (using underscores, as hyphens are illegal inside Python `import` statements)

## Initializing your Third Repository

To set this up on GitHub and tie it directly into your local agent's workflow, create your repository as `nlp-policy-nz`. Inside the root folder, establish the following files to turn it into an installable, shared core package:

### 1. `pyproject.toml` (The Dependency Blueprint)

Place this file in the absolute root directory of your new repository. It tells pip exactly what libraries your unified core needs to run.

toml

```
[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"

[project]
name = "nlp_policy_nz"
version = "0.1.0"
description = "Unified spaCy and Hugging Face NLP engine for New Zealand"
readme = "README.md"
requires-python = ">=3.10"
dependencies = [
 "spacy>=3.7.0",
 "transformers>=4.40.0",
 "datasets>=2.19.0",
 "pyarrow>=16.0.0"
]
```

Use code with caution.

### 2. The Internal Package Directory

Create a subfolder named exactly `nlp_policy_nz`. Inside it, create an empty file named `__init__.py`. This tells Python that this directory is a module that can be imported.

Your folder structure will now look like this:

text

```
nlp-policy-nz/ (GitHub Repo Root)
├─ pyproject.toml
```

```
└─ nlp_policy_nz/ (Internal Package Root)
 └─ __init__.py
 └─ core_pipeline.py # Your local agent will write this script
 └─ rules.py # For Te Reo Māori and NZ Act regex rules
```

Use code with caution.

## Updated XML Task Protocol for Your Local Agent

Now that you have defined the exact name and architecture, you can give your local coding agent its final instructions to write the code package.

**Pass this directly to your local agent:**

xml

```
<instruction_packet task="initialize_shared_core_package">
 <context>
 We have named our third repository 'nlp-policy-nz'. The code r
 </context>

 <technical_stack>
 - Python 3.10+
 - Hugging Face `datasets` (streaming mode)
 - `spacy` v3.x+
 - `transformers` with fast Rust tokenizers ('nlpaueb/legal-bei
 </technical_stack>

 <pipeline_specifications>
 Inside the 'nlp_policy_nz/' directory, implement the core pipe
 1. Accepts a streaming Hugging Face iterable dataset.
 2. Configures a blank spaCy pipeline with a sentence segmenter
 3. Customizes the spaCy tokenizer to protect Te Reo Māori terr
 4. Processes text in parallel batches and yields a standardiz
 - `doc_id`
 - `corpus_source` (legislation or hansard)
 - `raw_text`
 - `cleaned_tokens`
 - `sentence_count`
 </pipeline_specifications>

 <delivery_format_protocol>
 You must respond using the following XML tags:
```

<architecture\_plan>

Explain how the 'nlp\_policy\_nz' module handles streaming (

</architecture\_plan>

<unified\_core\_code>

Provide the complete code for 'nlp\_policy\_nz/core\_pipeline'

</unified\_core\_code>

<schema\_validation>

Display a Markdown table showing how the resulting schema

</schema\_validation>

</delivery\_format\_protocol>

</instruction\_packet>

Use code with caution.

Are you planning to push this `nlp-policy-nz` repository live onto GitHub right away, or would you like to verify the tokenization behavior of the **Te Reo Māori exceptions** locally first?

